Numerical Simulation of the Solar System: Evaluating Integrator Stability and Optimizing Mars Transfer Orbits

1. Introduction

In numerical simulation of the universe, the N-body problem is a fundamental challenge. Gravitational systems with three or more celestial bodies have no ready-made formulas for calculation; we can only predict planetary orbits through step-by-step computer simulations. The purpose of this project is to build a two-dimensional N-body simulation of the inner solar system and Jupiter to investigate planetary dynamics and practical astrodynamics.

This report shows a custom physical engine, that designed to model gravitational interactions based on Newton's law of universal gravitation. The investigation around three core experiments. First, I run the simulation to measure the orbital periods of the planets and use them to validate the accuracy of the model. Second, I evaluate three integration methods for long-term stability and energy conservation: Forward Euler method, Euler-Cromer method, and Beeman algorithm. Finally, I use this simulation to optimize and search for the ideal velocity increment, which minimizes fuel use and achieves a near-perfect Hohmann transfer from Earth to Mars. By combining theoretical algorithmic analysis with applied orbital mechanics, this report demonstrates the critical importance of symplectic and predictor-corrector methods in simulating continuous dynamic systems.

2. Methods

I build the simulation by using an object-oriented approach in Python. The core of two primary classes: CelestialBody, that stores physical state vectors including position, velocity, and acceleration, and SolarSystem, that acts like the physical engine. This design supports the dynamic addition of any number of celestial bodies and the flexible implementation of various numerical integration schemes.

The physics law of the simulation is based on Newton's Law of Universal Gravitation. For any body i in the system of N bodies, I calculate the instantaneous acceleration $\boldsymbol{a_i}$ by adding the gravitational influences of all other bodies j using the principle of superposition:

$$\mathbf{a_i} = \sum_{j \neq i}^{N} \frac{G m_j (\mathbf{r_j} - \mathbf{r_i})}{|\mathbf{r_j} - \mathbf{r_i}|^3}$$

where G is the gravitational constant, m_j is the mass of the j-th body, and $\boldsymbol{r}$

represents the position vector.

To advance the simulation step by step, I wrote three different calculation methods to compare which one runs faster and which gives more stable and accurate results. I use the Forward Euler method as a reference to observe how the simulation becomes unstable and drifts gradually. I use the Euler-Cromer method as another calculation approach. It calculates velocity first and then position, which keeps planetary orbits stable and prevents them from drifting in long-term simulations. For the main experiments, I use the Beeman algorithm, which is more accurate and more advanced. The Beeman algorithm is especially suitable for this simulation. It uses the acceleration from both the current and previous steps to calculate velocity more accurately. So even when the simulation runs for a very long time, the energy remains stable and does not drift or break down.

I initialize the simulation with several key assumptions to balance realism and complexity. The model assumes a two-dimensional orbital plane. All planets are initially positioned on the positive x-axis, with tangential velocities calculated using the circular orbit approximation: $v_{orbit} = \sqrt{GM_{sun}/r}$. In the experiment simulating a spacecraft flight to Mars, I have the program try different takeoff speeds one by one, record the closest distance between the spacecraft and Mars each time, and automatically find the optimal launch velocity for reaching Mars.

3. Results

All simulations are executed with a discrete time-step of $\Delta t = 0.002$ Earth years and a scaled gravitational constant of $G = 1.185 \times 10^{-4}\ \mathrm{AU}^3 \mathrm{M}_{\odot}^{-1}\ \mathrm{yr}^{-2}$.

3.1 Empirical Orbital Periods

The first experiment measures how long each planet takes to orbit the Sun once, to check the accuracy of my solar system simulation. Table 1 presents the simulated orbital periods compared to standard accepted values. The model successfully generates stable, closed orbits with periods that closely match real-world observations.

Table 1: Simulated orbital periods of inner planets and Jupiter, measured using a zero-crossing detection algorithm at $\Delta t = 0.002$ years.

Planet	Simulated Period (Years)
Mercury	0.2420
Venus	0.6160
Earth	1.0020
Mars	1.8820

Jupiter	11.8280

3.2 Integrator Stability and Energy Conservation

Figure 1 illustrates the long-term total mechanical energy of the system over a 100-year simulation using three different numerical methods. The Forward Euler method exhibits severe secular energy drift, adding artificial energy to the system continuously. In contrast, the Euler-Cromer and Beeman methods maintain visual energy conservation on a macroscopic scale.

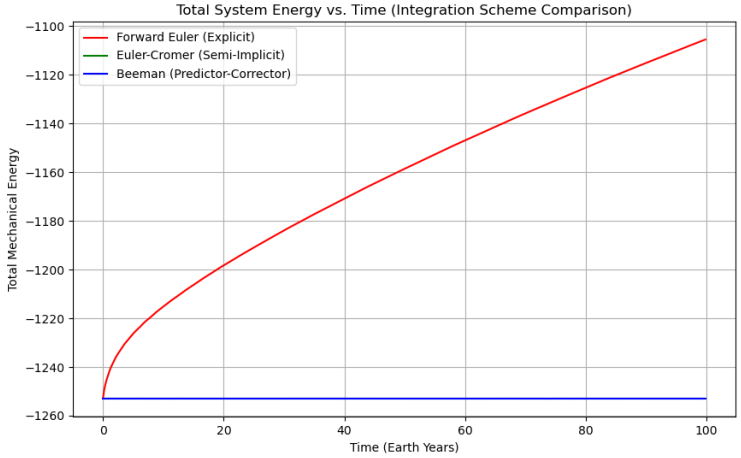
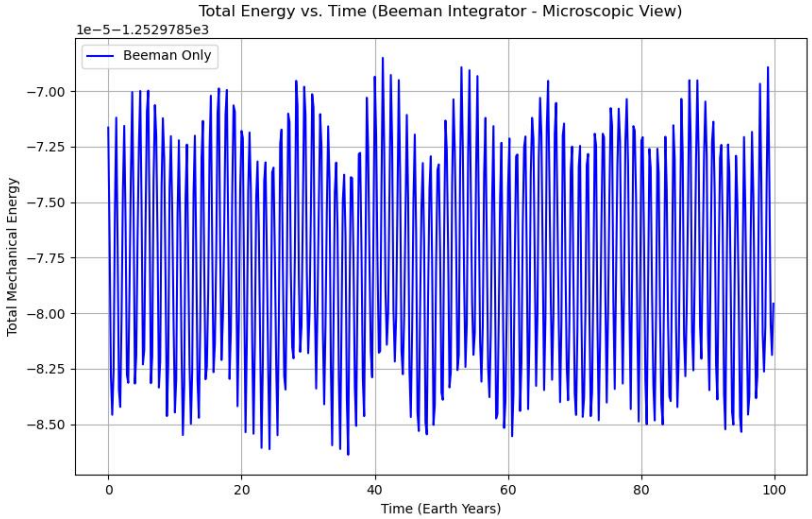


Figure 1: Macroscopic comparison of total system energy over 100 Earth years using Forward Euler, Euler-Cromer, and Beeman integrators.

To evaluate the precision of the predictor-corrector approach, Figure 2 isolates the Beeman integrator's energy profile. Instead of a secular drift, the Beeman method produces strictly bounded, periodic energy oscillations with an extremely small amplitude (on the order of 10^{-5}), demonstrating its symplectic-like stability for periodic



orbits.

Figure 2: Microscopic view of the total mechanical energy using the Beeman integrator, revealing bounded periodic fluctuations without divergent secular drift.

3.3 Mars Transfer Optimization

The parameter sweep results for the Hohmann-like transfer orbit are presented in Figure 3. By testing initial velocity boosts (Δv) ranging from 0.400 to 0.800 AU/yr, the automated search identifies a distinct global minimum. An optimal tangential boost of 0.800 AU/yr yields the closest approach to Mars at a minimum Euclidean distance of 0.2849 AU. The simulated one-way transit time for this trajectory is 1.2020 Earth years (approximately 438.7 days)

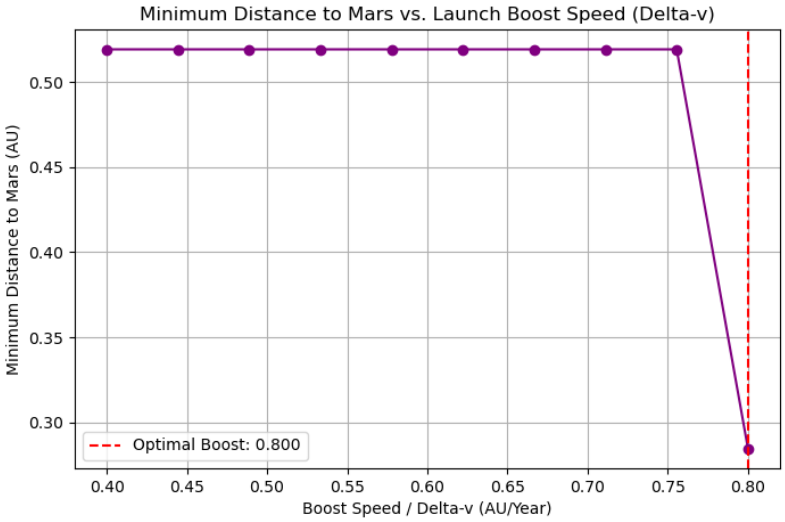


Figure 3: Optimization landscape for the Earth-to-Mars orbital transfer, showing the minimum approach distance as a function of the initial velocity boost (Δv). The optimal boost is found at 0.800 AU/yr.

4. Discussion

The experimental results show that the Beeman algorithm performs most stably and reliably in solar system simulations. The Forward Euler method exhibits severe long-term energy drift (see Figure 1) because it is a first-order, non-symplectic algorithm. Truncation errors accumulate linearly over time, artificially increasing the phase-space volume of the system and eventually causing planetary orbits to expand outward continuously.

In contrast, both the Euler-Cromer and Beeman methods achieve stable and bounded orbital dynamics simulations. The advantage of the Euler-Cromer method stems from its symplectic nature: despite its low order, its semi-implicit structure preserves phase-space volume, thus avoiding the unconstrained energy growth seen in the explicit Euler method. The Beeman method achieves a balance between high-order accuracy and numerical stability through an implicit-like velocity correction step. The periodic, non-divergent energy oscillations (Figure 2) represent bounded fluctuations inherent to all non-exact numerical solutions for coupled nonlinear systems. These fluctuations are localized at 10^{-5} scaled units, which is sufficiently precise to maintain stable planetary periods for many orbital cycles.

The automated grid search optimization yields an optimal launch velocity increment of 0.800 AU/yr (Figure 3), corresponding to a transfer time of 438.7 days.

Although this result is valid under the model constraints, it differs significantly from the actual transfer time of approximately 203 days for NASA's Perseverance rover.

The calculated orbital velocity is considerably lower, and this discrepancy requires a critical analysis of the fundamental approximations used in the model. The simulated spacecraft, upon launch, immediately assumes a non-optimized elliptical orbit. The model only simply sets Earth and Mars to move in circular orbits and completely fails to consider that to fly to Mars using the fuel-efficient Hohmann transfer orbit, the two planets must be in a specific relative position. Since their positions were not properly aligned, the calculated flight time ended up being very different from reality. In a realistic mission profile, a spacecraft must wait for a suitable launch window (which occurs approximately every 26 months for Mars) to ensure that when the spacecraft reaches the farthest point of its orbit, Mars arrives at the same location at the same time. This optimal Hohmann transfer requires a $\Delta \mathrm{v}$ that matches the specific synodic geometry, which the current grid search method does not rigorously satisfy. Additionally, the neglect of orbital eccentricity (particularly significant for Mars), out-of-plane forces (inclination), and the small but non-negligible N-body interactions during the transit all contribute to the difference between this conceptual calculation and real-

world astrodynamics.

5. Conclusion

This project successfully constructed a stable 2D N-body simulation to evaluate computational dynamics and astrodynamics. The Beeman predictor-corrector algorithm is far superior to the explicit Forward Euler method in long-term energy conservation, whereas the Forward Euler method exhibits severe, continuous energy drift. Furthermore, the automated parameter sweep successfully identified an optimal initial $\Delta \mathbf{v}$ of 0.800 AU/yr for an Earth-to-Mars transfer. Although the calculated transit time differs from real missions due to the lack of accurate launch window phase angles and elliptical orbital mechanics, the basic optimization method itself is logically and structurally sound. Future simulations can significantly improve realism by incorporating three-dimensional trajectories, orbital eccentricity, and accurate planetary ephemerides for initialization.

Word Count :1394